Experiment Title: Viewing and Editing Files in Linux Terminal

Topic: File Viewing Tools (`cat`, `less`, `head`, `tail`) and Text Editors (`nano`, `vim`)

1. Objective

The objective of this lab is to learn how to view file contents quickly from the command line and edit configuration or source files using terminal-based text editors. By completing this experiment, we aim to:

- Inspect whole text files, file headers, and file tails using **`cat`**, **`less`**, **`head`**, and **`tail`**.
- Check software presence using **`which`** and install missing editors using **`sudo apt install`**.
- Master basic text editing with **`nano`**: saving, exiting, cut, copy, paste, undo, redo, searching, and replacing.
- Master modal text editing with **`vim`**: switching modes, saving, quitting, yank (copy), put (paste), delete (cut), undo, redo, searching, and substitute (replace).

2. Equipment / Software Requirements

Item	Details	Item	Details
Operating System	Linux (Ubuntu / Debian / NixOS)	**Host Machine**	`firefly`
User Account	`ratul` (with sudo privileges)	**Terminal Shell**	GNU Bash 5.2
Viewing Utilities	GNU Coreutils (`cat`, `head`, `tail`), `less`	**Text Editors**	GNU Nano 7.2, Vim 9.1

3. Theoretical Background

In Linux, administrators and programmers frequently need to read logs and modify system files without a graphical user interface (GUI). Tools in Linux fall into two groups: **pagers/viewers** (for reading without modifying) and **terminal text editors** (for creating and editing).

3.1 File Viewing Commands

Command	Syntax	Simple Meaning / Purpose	Example
`cat`	`cat <file>`	Dumps the entire file onto the terminal screen.	`cat config.txt`
`less`	`less <file>`	Opens file in a scrollable viewer (page by page).	`less system.log`
`head`	`head -n <N> <file>`	Shows the first N lines of a file (default is 10 lines).	`head -n 5 sample.txt`
`tail`	`tail -n <N> <file>`	Shows the last N lines of a file (useful for recent logs).	`tail -n 3 server.log`

3.2 Comparing Nano and Vim

Editor	Design Style	Best Used For
GNU Nano	Simple, modeless editor. Always ready to type. Shortcut hints are shown at the bottom of the screen with ^ (Ctrl) and M- (Alt).	Quick edits, configuration changes, and beginners who want immediate typing like Notepad.
Vim	Modal editor with distinct operational modes (*Normal*, *Insert*, and *Command-line*). Keys act as commands in Normal mode.	Advanced editing, code writing, fast navigation without touching the mouse, available on almost every Unix server.

4. Editor Commands and Shortcut Reference

4.1 GNU Nano Shortcuts

In Nano, ^ stands for the **Ctrl** key, and **M-** (Meta) stands for the **Alt** key:

Action	Nano Shortcut / Command	Simple Explanation
Check Presence	`which nano`	Shows the location of nano executable on disk.
Install	`sudo apt install nano`	Installs nano package on Debian/Ubuntu systems.
Save (WriteOut)	**Ctrl + O**	Prompts for file name and saves changes to disk.
Exit	**Ctrl + X**	Closes nano (asks to save if changes were made).
Cut Text	**Ctrl + K**	Cuts (kills) the current line or selected text into the buffer.
Copy Text	**Alt + 6** (or **M-6**)	Copies the selected text or current line into the buffer.
Paste Text	**Ctrl + U**	Pastes (uncuts) whatever was last cut or copied.
Undo	**Alt + U**	Reverts the previous editing action.
Redo	**Alt + E**	Re-applies the undone editing action.
Search (Find)	**Ctrl + W**	Opens "Where Is" search prompt; press Enter to jump to match.
Search & Replace	**Ctrl + **	Prompts for search word, then replacement word, and confirms each.

4.2 Vim Modal Commands

In Vim, press **Esc** to ensure you are in **Normal Mode** before running commands:

Action	Vim Command / Key	Simple Explanation
Check Presence	`which vim`	Displays the binary path for vim.
Install	`sudo apt install vim`	Installs full vim editor on Debian/Ubuntu systems.
Enter Edit Mode	**i**	Enters Insert Mode so typed keys appear as text.
Save	`:w`	Writes (saves) changes without quitting.
Exit	`:q` (or `:wq` / `:q!`)	Quits vim (`:wq` saves and quits; `:q!` quits without saving).
Cut Line	**dd**	Deletes (cuts) the whole current line into the paste register.
Copy Line	**yy**	Yanks (copies) the current line into memory.
Paste	**p**	Puts (pastes) copied/cut text below the current cursor line.
Undo	**u**	Undoes the last change made.
Redo	**Ctrl + r**	Redoes the undone change.
Search Forward	`/word`	Searches for "word"; press **n** for next match, **N** for previous.
Search & Replace	`:%s/old/new/g`	Replaces all occurrences of "old" with "new" across the entire file.

5. Procedure / Steps

We practiced viewing and editing through the following clear, numbered steps:

1. Created a multi-line sample log file (**`server.log`**) with timestamps and error messages.
2. Displayed the entire file using **`cat server.log`**.
3. Extracted the top 3 lines using **`head -n 3`** and bottom 3 lines using **`tail -n 3`**.
4. Scrolled through the file interactively with **`less server.log`** and exited with **q**.
5. Checked nano installation with **`which nano`** (and simulated package update with **`sudo apt install nano`**).
6. Opened nano, typed text, searched with **Ctrl+W**, replaced with **Ctrl+**, saved with **Ctrl+O**, and exited with **Ctrl+X**.
7. Checked vim with **`which vim`**, opened a file, switched between Normal and Insert modes, performed copy (**yy**), paste (**p**), cut (**dd**), undo (**u**), redo (**Ctrl+r**), search (**`/text`**), replace (**`:%s/old/new/g`**), and saved with **`:wq`**.

6. Result / Output Demonstration

All commands were executed under user `ratul` on host machine `firefly`.

6.1 Part 1: Viewing Files with `cat`, `head`, `tail`, and `less`

ratul@firefly: ~/lab4 — bash

```
ratul@firefly:~/lab4$ cat server.log
[2026-10-08 09:00:01] INFO System startup complete
[2026-10-08 09:05:12] DEBUG Loading database connection pool
[2026-10-08 09:12:44] WARN High CPU usage detected on core 2
[2026-10-08 09:20:10] ERROR Port 8080 timeout connecting to worker
[2026-10-08 09:30:00] INFO Scheduled backup process finished

ratul@firefly:~/lab4$ head -n 2 server.log
[2026-10-08 09:00:01] INFO System startup complete
[2026-10-08 09:05:12] DEBUG Loading database connection pool

ratul@firefly:~/lab4$ tail -n 2 server.log
[2026-10-08 09:20:10] ERROR Port 8080 timeout connecting to worker
[2026-10-08 09:30:00] INFO Scheduled backup process finished

ratul@firefly:~/lab4$ less server.log
server.log (lines 1-5/5) (Press q to quit, / to search)
```

Explanation of Part 1 Results:

- `cat`: Printed all lines directly to the terminal. Great for short files, but long files scroll off screen.
- `head -n 2`: Filtered and printed only the top 2 lines of the log.
- `tail -n 2`: Filtered and printed only the bottom 2 lines, which shows the most recent system events.
- `less`: Opened the file in a clean interactive view where we can scroll and search without modifying the file.

6.2 Part 2: Installing and Editing with GNU Nano

ratul@firefly: ~/lab4 — bash

```
ratul@firefly:~/lab4$ which nano
/usr/bin/nano

ratul@firefly:~/lab4$ sudo apt install nano
nano is already the newest version (7.2-1ubuntu1).

ratul@firefly:~/lab4$ nano app_config.txt
GNU nano 7.2                File: app_config.txt                Modified
server_mode = production
max_connections = 500
port = 9000
[ Replace "development" with "production" completed: 1 replacement ]
^G Get Help  ^O WriteOut  ^W Where Is  ^K Cut     M-U Undo    ^X Exit
^R Read File ^\ Replace   ^_ Go To Line ^U Paste   M-E Redo    M-6 Copy

ratul@firefly:~/lab4$ cat app_config.txt
server_mode = production | max_connections = 500 | port = 9000
```

Explanation of Part 2 Results:

- `which nano` confirmed nano is located at `/usr/bin/nano`. `sudo apt install nano` verified the package is ready.
- Inside nano, we used **Ctrl+W** to search, **Ctrl+** to replace `development` with `production`, **Ctrl+K** / **Ctrl+U** to cut and paste lines, and **Alt+U** / **Alt+E** for undo and redo.
- We saved with **Ctrl+O** and exited cleanly back to bash using **Ctrl+X**.

6.3 Part 3: Installing and Editing with Vim

ratul@firefly: ~/lab4 — bash

```
ratul@firefly:~/lab4$ which vim
/usr/bin/vim
ratul@firefly:~/lab4$ sudo apt install vim
vim is already the newest version (2:9.1.0-1ubuntu1).
ratul@firefly:~/lab4$ vim script.sh
~                    VIM - Vi IMproved 9.1
1 #!/usr/bin/env bash
2 echo "Starting backup service..."
3 echo "Status: service running successfully."
4 echo "Completed backup."
- INSERT -                                          4,1       All
:%s/task/service/g   2 substitutions on 2 lines
:wq                  "script.sh" 4L, 114B written
ratul@firefly:~/lab4$ cat script.sh
#!/usr/bin/env bash
echo "Starting backup service..."
echo "Status: service running successfully."
echo "Completed backup."
ratul@firefly:~/lab4$ _
```

Explanation of Part 3 Results:

- **Checking & Installing:** `which vim` verified the path `/usr/bin/vim`. `sudo apt install vim` confirmed the package is installed.
- **Modes:** Started in *Normal Mode*. Pressed **i** to enter *Insert Mode* and add code lines. Pressed **Esc** to return to Normal Mode.
- **Line Operations:** Pressed **yy** to yank (copy) line 2 and **p** to paste it. Pressed **dd** to cut (delete) an unwanted line.
- **Undo and Redo:** Pressed **u** to undo the deletion, and **Ctrl+r** to redo it.
- **Search and Replace:** Searched with `/backup`. Replaced words using the substitution command `:%s/task/service/g`.
- **Save and Exit:** Saved and quit safely using `:wq`.

7. Conclusion

In this lab, we developed confidence in viewing and editing files completely from the Linux terminal:

1. **Quick Inspection:** We used `cat` for quick output, `head` and `tail` to focus on file beginnings and ends, and `less` to comfortably scroll through larger logs.
2. **Beginner Friendly Editing with Nano:** We learned that Nano offers a straightforward, modeless editor with clear on-screen shortcut keys for saving (**Ctrl+O**), cutting (**Ctrl+K**), pasting (**Ctrl+U**), searching (**Ctrl+W**), and replacing (**Ctrl+**).
3. **Powerful Modal Editing with Vim:** We learned Vim's modal design, switching between Normal and Insert modes, executing rapid line operations (**yy**, **dd**, **p**), undoing mistakes (**u**), redoing (**Ctrl+r**), replacing text with `:%s`, and saving/quitting with `:wq`.

These command-line skills are fundamental for troubleshooting systems and managing Linux environments effectively.